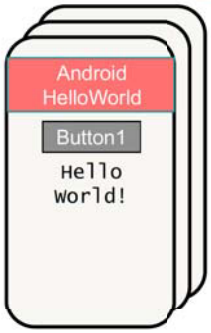
PLAN

- 1. Introduction programmation mobile (Android)
- 2. Cycle de Vie d'une Activité
- 3. Environnement de Développement
- 4. Interfaces graphiques
- 5. Les Intents
- 6. Persistance des données
- 7. Capteurs sous Android
- 8. Programmation Concurrente

Les Intents

- Une application Android peut contenir plusieurs activités :
  - Une activité utilise la méthode setContentView pour s'associer avec une interface graphique
  - Les activités sont indépendantes les unes des autres, cependant, elles peuvent collaborer pour échanger des données et des actions
  - Une des activités est appelée l'activité de démarrage, c'est la première à être présentée à l'utilisateur quand l'application est lancée.
  - Le passage d'une activité à une autre est réalisé en demandant à l'activité en cours d'exécuter un Intent.



Les Intents

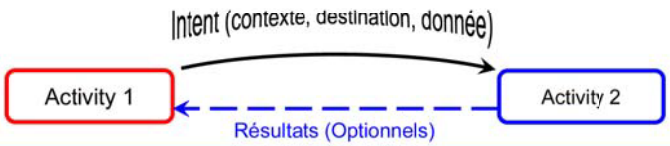
- Un Intent est un message qui peut être utilisé pour demander une action à partir d'un autre composant de l'application
- Un Intent permet invoquer des Activités, des Broadcast Receivers ou des Services.
- Les différentes méthodes utilisées pour appeler ces composantes sont les suivantes :
  - startActivity (intent) : lance une activité
  - sendBroadcast (intent) : envoie un intent à tous les composants Broadcast Receivers intéressés
  - startService (intent) ou bindService (intent, …) : communiquent avec un service en arrière plan.

Les Intents

- Il existe deux types d'Intents
  - Intents Explicites:
    - Spécifient le composant à démarrer par nom de la classe
    - Permettent de démarrer un composant de l'application, car le nom de la classe est connu
    - Exemple: démarrer une activité en réponse à l'action d'un utilisateur
  - Intents Implicites
    - Déclarent une action à réaliser sans nommer un composant spécifique
    - Le système lancera l'activité la plus apte à satisfaire la requête
    - Permet à un composant d'appeler un composant d'une autre application
    - Exemple : montrer à l'utilisateur un emplacement sur une Map

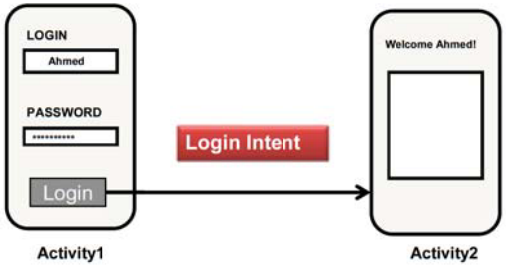
Les Intents : Intents Explicites

- Les principaux arguments d'un Intent explicite sont :
  - Le contexte déclenchant l'Intent :
    - En général this, si on le lance à partir de l'activité de départ,
    - ou bien <Activity_class_name>.this)
  - La classe destination (en général <Activity_class_name>.class)
- Il est typiquement appelé comme suit:
  - Intent myActivityIntent = new Intent (<context>, <destination>) ;
  - startActivity (myActivityIntent) ;



Les Intents : Intents Explicites : Exemple

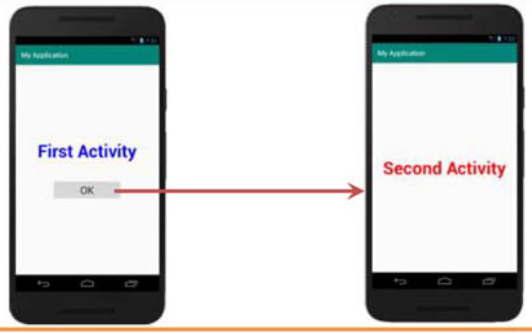
- Activité d'Authentification vers Activité d' Accueil



Les Intents : Intents Explicites

- Cible une activité particulière à lancer
  - Il est typiquement appelé comme suit:

```
Intent myIntent = new Intent(StartClass.this, EndClass.class);
startActivity(myIntent);
```



Les Intents : Intents Explicites

- Transmettre des données : les Intents peuvent aussi transmettre des données d'une activité à une autre



Les Intents : Intents Explicites

- Transmission des données : Les Intent peuvent aussi transmettre des données d'une activité à une autre

Activité appelante : intent.putExtra(key.value);

```
btn = findViewById(R.id.btn);
edtNom = findViewById(R.id.nom);
edtPrenom = findViewById(R.id.prenom);

btn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent myIntent = new Intent(StartClass.this,EndClass.class);
        myIntent.putExtra("lName",edtNom.getText().toString());
        myIntent.putExtra("fName",edtPrenom.getText().toString());
        startActivity(myIntent); }
});
```


Les Intents : Intents Explicites

- Récupération des données : Utilisation de la clé pour l'adressage

Activité appelée: intent.getXXXExtra(key);

```
Intent intentRecup= getIntent();
String nom= intentRecup.getStringExtra("lName");
String prenom= intentRecup.getStringExtra("fName");

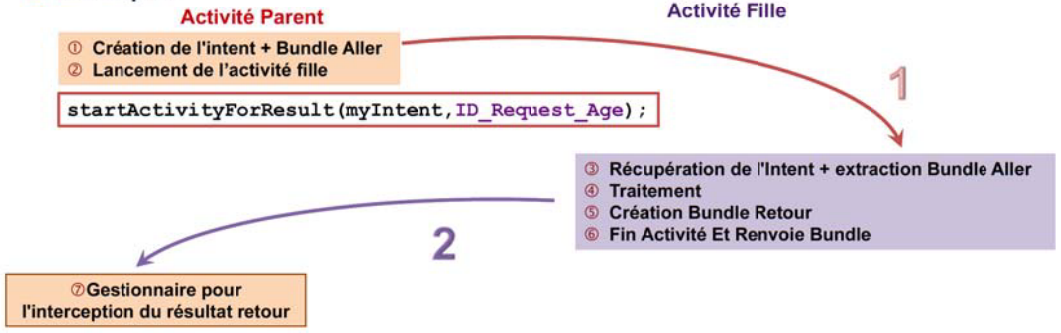
txt= findViewById(R.id.txt);
txt.setText("Hello "+ prenom + " " + nom);
```


Les Intents : Intents Explicites

- Il est possible d'établir un lien bidirectionnel entre deux activités grâce à un Intent
- Pour recevoir un résultat à partir d'une autre activité, appeler startActivityForResult au lieu de startActivity
- L'activité destination doit bien sûr être conçue pour renvoyer un résultat une fois l'opération réalisée.
- Le résultat est envoyé sous forme d'Intent.
- L'activité principale le recevra dans la méthode onActivityResult

Les Intents : Intents Explicites

- L'activité appelante (parent) démarre l'activité appelée (fille)
- Le parent attend un résultat en retour
- Les étapes :




```
protected void onActivityResult(int requestCode, int resultCode, Intent data)
```

Les Intents : Intents Explicites

- Exemple : Activités avec retour de résultats



Les Intents : Intents Explicites

- Démarrer une Activité avec Résultat

Activité appelante : `startActivityForResult`

```
btn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent myIntent = new Intent(StartClass.this,EndClass.class);
        myIntent.putExtra("lName",edtNom.getText().toString());
        myIntent.putExtra("fName",edtPrenom.getText().toString());
        startActivityForResult(myIntent,ID_Request_Age);
    }
});
```


Les Intents : Intents Explicites

- Récupération de l'Intent + extraction « Bundle Aller»
- Création Bundle Retour

```
Intent intentRecup= getIntent();
String nom= intentRecup.getStringExtra("lName");
String prenom= intentRecup.getStringExtra("fName");
txt.setText("Hello "+ prenom + " " + nom);
btnVal.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
     Intent myIntentResult = new Intent();
     myIntentResult.putExtra("result",edtAge.getText().toString());
     setResult(RESULT_OK,myIntentResult);
     finish();  }    });
btnAnn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent myIntentResult = new Intent();
        setResult(RESULT_CANCELED);
        finish();   }    });
```


Les Intents : Intents Explicites

- Gestionnaire pour l'interception du résultat retour

```
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    if(requestCode==ID_Request_Age){
      if(resultCode==RESULT_OK)
         txtAffResult.setText("votre age est " + data.getStringExtra("result"));
      else
         txtAffResult.setText("pas de saisie");
    }
}
```

Les Intents : Intents Explicites

- **int requestCode : valeur identifiant quelle activité a appelé la méthode.**
  - Cette même valeur a été spécifiée comme paramètre de la méthode startActivityForResult lors du démarrage de la sous-activité ;
- **int resultCode : représente la valeur de retour envoyée par la sous-activité pour signaler son état à la fin de la transaction.**
  - C'est une constante définie dans la classe Activity (RESULT_OK, RESULT_CANCELED, etc.) ou par le développeur ;

Les Intents : Intents Explicites

- **erreur du type ActivityNotFoundException**



- **déclarer l'activité à démarrer dans le fichier de configuration AndroidManifest.xml de l'application**

Ne pas oublier de déclarer toutes les activités de votre application dans Manifest

Les Intents : Intents Explicites : Résumer

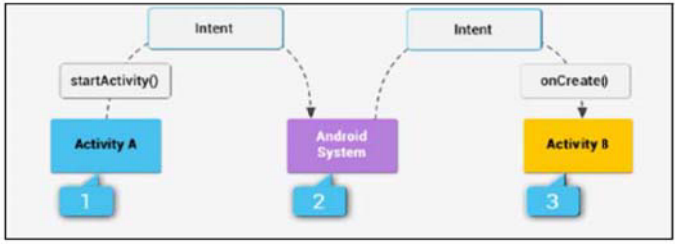
- **Intent.putExtra() :ajouter des données supplémentaires :**
  - Clé (String) => valeur (types primitifs (int, float,..), String, Bundle, …)
- **L'activité appelée peut connaitre l'intent appelant *getIntent()*, l'action de l'intent *getAction()*, la donnée (uri) *getData()* et les données extras *getExtras()***
  - **Bundle extras = getIntent().getExtras();**
  - **String valeur = extras.getString("clé_envoyée");**
- **finish(): finish() fait terminer l'activité courante. L'utilisateur ne pourra pas faire back dessus, car elle disparaît de la pile.**
- **Lancer l'activité :**
  - **startActivity(intent) : appel d'une activité qui peut être externe à l'application**
  - **startActivityForResult(intent, coderesultat) : appel une sous activité et reste en attente de résultat :**
    - **La sous activité peut retourner un code de résultat : setResult(coderesultat) avant le finish() (par défaut on a RESULT_OK; RESULT_CANCELED,)**
    - **La sous activité peut retourner des données supplémentaires sous forme d'intent**
    - **Une fois la sous-activité terminée (finished), la méthode onActivityResult() est appelée par l'activité mère**

Les Intents : Intents Implicites

- **Les principaux arguments d'un Intent implicite sont :**
  - **Nom du composant à démarrer**
  - **Action : l'action à réaliser, peut être prédéfinie (ACTION_DIAL, ACTION_VIEW, ACTION_SENDTO, ACTION_PICK, ACTION_CALL …) ou créée par l'utilisateur.**
  - **Donnée : URI référençant la donnée sur laquelle l'action va agir, tel que le numéro de téléphone à appeler.**
  - **Catégorie : Information supplémentaire sur le type de composants qui va gérer l'intent : CATEGORY-BROWSABLE, CATEGORY-LAUNCHER…**
  - **Extras : Paires clef-valeur qui comportent des informations additionnelles pour réaliser l'action demandée**
  - **Drapeaux (Flags) :**
    - **Définissent la classe qui fonctionne comme métadonnée pour cet intent**
    - **Peuvent indiquer comment lancer une activité, comment la traiter une fois lancée**

Les Intents : Intents Implicites

- Un intent implicite se comporte comme suit:
  1. Activité A créé un Intent avec une action et le passe en paramètre à startActivity
  2. Le système Android cherche toutes les applications pour trouver un Intent filter qui correspond à cet Intent
  3. Quand une correspondance est trouvée, le système démarre l'activité (Activity B) en invoquant sa méthode onCreate et en lui passant l'intent



Les Intents : Intents Implicites

- Un Intent filter est une expression dans le fichier Manifest d'une application qui spécifie le type d'intents que le composant veut recevoir
- Permet aux autres activités de lancer directement une activité en utilisant une instance de Intent
- Si vous ne déclarez pas d'Intent filters à votre activité, elle ne pourra être déclenchée que par un Intent Explicite
- Il est recommandé de ne pas déclarer d'Intent Filters pour les services, car cela peut causer des problèmes de sécurité

```
<activity android:name=".MainActivity">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```


Les Intents : Intents Implicites : Exemples

- URI : Uniform Resource Identifier, qui signifie Identifiant de ressource uniforme. Généralise la notion d'URL
- URL : Uniform Ressource Locator, est une information permettant de localiser un élément. L'adresse d'un site internet est une URL.
- La methode Uri.parse permet de créer un Uri object a partir de d'une chaine String
- Exemple 1 : appeler un n° de téléphone

```
b.setOnClickListener(new OnClickListener() {
@Override
public void onClick(View v) {
        Uri telnumber = Uri.parse("tel:0248484000");
        Intent call = new Intent(Intent.ACTION_DIAL, telnumber);
        startActivity(call);     }         });
```

- Exemple 2 : visualiser une page internet

```
Uri chemin = Uri.parse("http://www.google.fr");
Intent naviguer = new Intent(Intent.ACTION_VIEW,chemin);
startActivity(naviguer);
```


Les Intents : Intents Implicites : Exemples

- Exemple 3 : visualiser une géolocalisation avec latitude/longitude

```
String latitude = "31.750996996";String longitude = "72.906329708";
Uri uriGeo = Uri.parse("geo:" + latitude + "," + longitude);
Intent intentGeo = new Intent(Intent.ACTION_VIEW, uriGeo);
startActivity(intentGeo);
```

Exemple 4 : choisir un contact parmi mon annuaire

```
Uri uri = Uri.parse("content://contacts/people");
Intent intent = new Intent(Intent.ACTION_PICK, uri);
startActivityForResult(intent,2);

// StartActivityForResult car une donnée est attendu en retour
```

Les Intents : Intents Implicites : Exemples

Exemple 5 : Envoi d'un mail. Dans ce cas il faut ajouter des informations supplémentaires à l'aide de putExtra

```
Uri uri = Uri.parse("mailto:myemail@gmail.com");
Intent intent = new Intent(Intent.ACTION_SENDTO,uri);
intent.putExtra("subject", "my subject");
intent.putExtra("body", "my message");
startActivity(intent);
```


Les Intents : Intents Implicites : Action

Action	Définition
ACTION_ANSWER	Prendre en charge un appel entrant.
ACTION_CALL	Appeler un numéro de téléphone. Cette action lance une activité qui affiche l'interface pour composer un numéro
ACTION_DIAL	Afficher l'interface de composition des numéros. Celle-ci peut être préremplie par les données contenues dans l'URI spécifiée en paramètre.
ACTION_EDIT	Éditer une donnée.
ACTION_SEND	Envoyer des données texte ou binaire par courriel ou SMS. Les paramètres dépendront du type d'envoi.
ACTION_SENDTO	Lancer une activité capable d'envoyer un message au contact défini par l'URI spécifiée en paramètre.
Action.MAIN	activité principale d'une application

Les Intents : Intents Implicites : Action

ACTION_SEARCH	Démarrer une activité de recherche. L'expression de recherche de la pourra être spécifié dans la valeur du paramètre SearchManager.QUERY envoyé en extra de l'action.
ACTION_VIEW	Démarrer une action permettant de visualiser l'élément identifié par l'URI spécifiée en paramètre **http**: lanceront un navigateur web, celles commençant par **tel**: lanceront l'interface de composation de numéro et celles débutant par **geo**: lanceront Google Map.
ACTION_WEB_SEARCH	Effectuer une recherche sur Internet avec l'URI spécifiée en paramètre comme requête.
ACTION_DELETE	Démarrer une activité permettant de supprimer une donnée identifiée par l'URI spécifiée en paramètre.

Les Intents : Intents Implicitespour Camera : Action

Des types d'Intent	Description
ACTION_IMAGE_CAPTURE_SECURE	Il renvoie l'**Image** capturée à partir de la **Caméra**, lorsque l'appareil photo est sécurisé
ACTION_VIDEO_CAPTURE	Il appelle l'application vidéo existante dans **Android** pour capturer une vidéo à partir de la **Caméra**.
EXTRA_SCREEN_ORIENTATION	Il sert à régler l'orientation de l'écran en **"vertical"** ou en **"landscape"** (paysage)
EXTRA_FULL_SCREEN	Il sert à contrôler l'interface utilisateur du **ViewImage**
INTENT_ACTION_VIDEO_CAMERA	Cette intention permet de lancer la **Caméra** en mode vidéo
EXTRA_SIZE_LIMIT	Il est utilisé pour spécifier la limite de taille de la taille de capture vidéo ou image.

Les Intents : intent filter

- **Le système peut choisir le composant le mieux adapté au traitement d'une action**
- **Chaque application peut facilement "écouter" les Intents dont elle a besoin**
- **Plusieurs niveaux de filtrage :**
  - action: identifie le nom de l'Intent. Pour éviter les collisions, il faut utiliser la convention de nommage de Java
  - category: permet de filtrer une catégorie d'action (DEFAULT, BROWSABLE, ...)
  - data: filtre sur les données du message par exemple en utilisant ancroid:host pour filtrer un nom de domaine particulier ou android:scheme (la partie gauche de l'url)

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme= "http" android:host= "ordiciel.ma"/>
</intent-filter>
```


Autres Composants Avancés

Services, Broadcast Receivers, Content Providers

https://devstory.net/10421/android-service

Autres Composants Avancés : Les services

- **De même que les Activités et les Intents, les services font partie des composants essentiels d'Android.**
  - Ils ne disposent pas d'interface utilisateur.
  - fonctionnent en arrière-plan pour une période de temps indéfinie

Autres Composants Avancés : Les services

- **Les applications qui s'exécutent fréquemment ou continuellement sans nécessiter une interaction avec l'utilisateur peuvent être implémentées sous la forme d'un service.**
- **Exemples : Les applications qui récupèrent des informations (informations, météo, résultats sportifs, etc.) via des flux RSS/Atom.**
  - Les flux RSS/Atom sont des formats de données basés sur XML utilisés pour diffuser automatiquement des mises à jour de contenu

Autres Composants Avancés : Les services

Il existe deux types de services :

- **Services locaux (LocalService) :**
  - Services qui s'exécutent dans **le même processus** que votre application.
  - Un service Local n'est accessible que par les Activités de l'application.
- **Services distants (RemoteService)**
  - s'exécutent dans un processus totalement différent de celui de votre application .
  - Un RemoteService peut être sollicité par plusieurs applications différentes.

Autres Composants Avancés : Les services

- Le service n'a pas de durée de vie définie, fonctionne tant que c'est nécessaire.
- Android fournit des services prédéfinis auxquels on peut accéder via la méthode *getSystemService()*.
- Tout autre service doit être déclarer dans le fichier AndroidManifest.xml via

  `<service android:name="yourclasss"> </service>`

Autres Composants Avancés : Les services

- **OnCreate()** : cette méthode est appelée à la création du service et est en général utilisée pour initialiser ce qui sera nécessaire à votre service ;
- **OnStartCommand (Intent i, int flags, int startId)** : le service démarre. Valable uniquement pour les versions du SDK supérieur à 2.0 ;
- **OnDestroy()** : appelé à la fermeture du service.



Autres Composants Avancés : Les services

- Pour créer un service, il suffit d'implémenter une classe qui hérite de la classe Service

```
public class MonPremierService extends Service
```

- Une activité peut démarrer un Service via la méthode startService() et le stopper via stopService()
- La méthode onBind sert quand on veut faire communiquer le service avec un autre service, ou avec une activité ..etc.

Autres Composants Avancés : Exemple service

- **Exemple : géolocalisation du téléphone**
  - Créer un **service** qui affichera un toast contenant les coordonnées du téléphone.
  - Le toast s'affichera quand les coordonnées seront mises à jour.

Autres Composants Avancés : Exemple service

- **On commence par créer une interface qui contient un bouton.**
- **Quand on clique sur le bouton, un toast s'affiche et nous indique les coordonnées du téléphone.**



Autres Composants Avancés : Exemple service

- **Création de l'interface :**

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    tools:context=".MainActivity">

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Lancer le Service de Géolocalisation"
        android:id="@+id/btnVal"/>
</LinearLayout>
```


Autres Composants Avancés : Exemple service

- **Dans la classe de l'activité principale, qu'on a nommé HomeActivity, nous allons:**
  - récupérer l'id du bouton
  - Créer un listener sur ce bouton
  - Démarrer le service quand ce bouton est cliqué

```
btnValider = findViewById(R.id.btnVal);

btnValider.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
      Intent serviceIntent = new Intent(MainActivity.this, MyService.class);
      startService(serviceIntent);
    }
});
```

Autres Composants Avancés : Exemple service

- Créer la classe service : MyService
- La classe MyService hérite de la classe Service
- On crée une instance de la classe LocationListener, pour récupérer les nouvelles coordonnées.

```
public class MyService extends Service {
    private LocationManager locationMgr = null;
    private LocationListener onLocationChange = new LocationListener()
{
```


Autres Composants Avancés : Exemple service

- Dans le onCreate, il faudra s'abonner au changement de géolocalisation à travers le service de localisation

```
public void onCreate() {

locationMgr = (LocationManager)getSystemService(Context.LOCATION_SERVICE);
locationMgr.requestLocationUpdates(LocationManager.NETWORK_PROVIDER, 10000, 0,
onLocationChange);

locationMgr.requestLocationUpdates(LocationManager.GPS_PROVIDER, 10000, 0,
onLocationChange);

}
```


Autres Composants Avancés : Exemple service

- requestLocationUpdates (String, long, float, LocationListener).
- Cette méthode possède 4 arguments :
  - Le provider utilisé pour recevoir les mises à jour des coordonnées utilisateurs (GPS / NETWORK …)
  - Un intervalle minimum entre deux notifications (en millisecondes)
  - Un intervalle minimum entre deux notifications (en mètre)
  - L'instance de votre LocationListener

Autres Composants Avancés : Exemple service

- Dans la méthode OnLocationChanged, nous allons afficher les coordonnées

```
public void onLocationChanged(Location location) {
    double latitude = location.getLatitude();
    double longitude = location.getLongitude();
    Toast.makeText(MyService.this, "voici les coordonées de votre
telephone " + latitude + " " + longitude, Toast.LENGTH_SHORT).show();
}
```

Autres Composants Avancés : Exemple service

- Maintenant, il faut déclarer le service dans le fichier AndroidManifest.xml, dans la partie Application

```
<service android:name=".MyService"/>
```

- Et les permissions de ce service:

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_GPS"/>
```


Autres Composants Avancés : Exemple service



Autres Composants Avancés

Services, **Broadcast Receivers, Content Providers**